
Where Does LLM Spatial Grounding Break? An Instruction-Tier Study for Goal-Conditioned Robotic Manipulation*

Joseph Warzybok Mckenney
UC San Diego
jwarzybokmckenney@ucsd.edu

Fong-Yu (Yang) Lin
UC San Diego
folin@ucsd.edu

Waleed Alghaithi
UC San Diego
walghaithi@ucsd.edu

Aditya Jadhav
UC San Diego
a6jadhav@ucsd.edu

Nasser Al Nasser
UC San Diego
nalnasser@ucsd.edu

Abstract

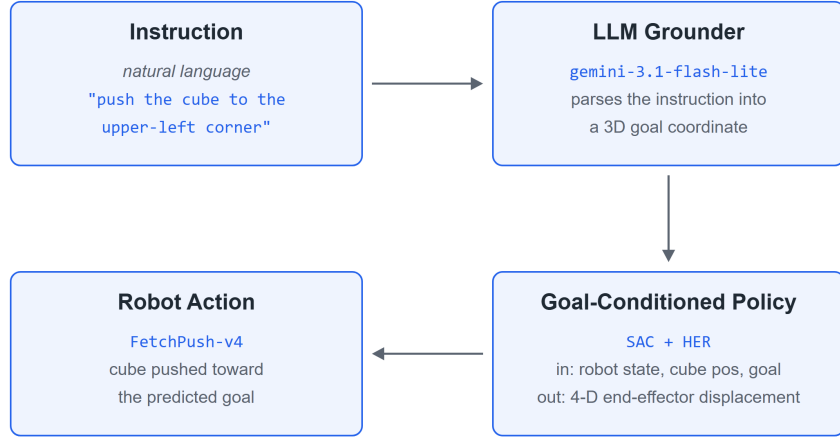
We present an empirical study of **what kind of error** LLM spatial grounding produces for goal-conditioned robotic manipulation, and which errors a downstream controller can recover from. We define a five-tier instruction hierarchy from explicit coordinates (T0) to functional intent (T4) and evaluate a gemini-3.1-flash-lite grounder paired with a pre-trained SAC+HER policy on FetchPush-v4. Across 100 evaluated instructions (20 per tier), accuracy declines with abstraction as hypothesized but the decline is the least interesting result. Each tier fails in a categorically different way, and that categorization, not the score, decides recoverability. T2 (relative offsets) is the dangerous tier: the LLM is worse than a regex baseline (40.0–65.0% vs 100.0% grounder success rate) and its errors are unrecoverable. T3 (reference objects) is the opposite: the grounder picks the right object and direction every time but introduces a 6 cm Y bias, a recoverable problem. T4 (functional intent) is not a grounding-accuracy problem at all; the model–annotator gap (0.20 m) matches the gap between annotators. A regex baseline confirms the LLM adds nothing at T0-T1 but is essential from T3 onward, where regex scores 0.0%, and prompt engineering produces minimal end-to-end differences ($\leq 17.0\%$). A controller adaptation experiment validates the recoverable/unrecoverable split: retraining drives policy SR to 100.0% everywhere, measurably tightening the T3 bias while doing nothing for T2’s off-table goals.

1 Introduction

When we’re talking to other people, we don’t ask them to go to specific coordinates, we say “move to the right a little,” or “move this out of the way.” The same logic should ideally apply when humans speak to robots. We seek to map these free-form instructions to the controller. The controller will execute a continuous task by calling an LLM, parsing JSON, and passing numbers to a policy. Modern LLMs can reliably produce well formatted output, however, they struggle with converting the natural language to the correct coordinates. In this paper, we use simulation experiments to explore where this conversion breaks down.

Our first assumption is that grounding gets monotonically worse as abstraction increases. Coordinate literals are the simplest, named regions come next, and broad or open-ended intent instructions

*Course project for CSE 190 at UC San Diego. This work has not been submitted to, reviewed by, or accepted at NeurIPS; the NeurIPS style file is used for formatting only.



The grounder decides *where* the cube should go; the policy decides *how* to push it there.

Figure 1: Overview of our language-to-goal grounding and controller evaluation pipeline.

are the hardest, with accuracy declining smoothly along the way. We organized instructions into five levels, from T0 (coordinate literals) to T4 (functional intent), and measured a frontier LLM’s grounding accuracy at each level with a fixed single uniform tolerance (5 cm). The accuracy does fall as abstraction increases, but the decline isn’t the interesting part, and we expected that. What we didn’t expect is that each level fails in a different way. T0 and T1 are essentially perfect. At T2 ($\sim 40 - 65\%$) the model often projects targets that fall outside the reachable workspace. At T3 ($0 - 5\%$) the model is consistently off by 6 cm in the Y direction, which is almost the same offset on every single instance. Therefore, accuracy becomes the wrong perspective: what really changes across levels is the character of the failure, and as we show next, that character is what decides whether a downstream controller can recover from it.

We investigated this problem on a MuJoCo operation task named FetchPush-v4, which is a 7-DOF arm that pushes a block to a target. We used a goal-conditioned policy trained with SAC + HER as the downstream controller, and gemini-3.1-flash-lite to do the coordinate conversions from natural language. We designed two experiments with different roles. Experiment 1 is the grounding with a fixed controller, which is the primary metric and the policy is trained only once on the target coordinates and then fixed as a neutral consumer. This lets end-to-end (E2E) success rate (SR) reflect the quality of grounding rather than the control ability. This shows the grounding accuracy at each layer, failure mode classification, and how grounding error spreads to task success. Experiment 2 focuses on the adaptation of the controller. It asks another question: Since the grounder generates systematic and layer specific errors, how much end-to-end (E2E) performance can be saved by having the controller absorb these failures? We define Experiment 2 to not incorporate its data into the grounding measurements of Experiment 1.

The recovery turns out to be very uneven, and that unevenness is actually the main point here. For T3, the grounder’s error is a consistent offset, it is off by 6 cm on the same axis almost every time. The adapted controller does perform more precise movements but cannot overcome the bias. For T2, the adaptation does basically nothing, because T2’s main failure is that the goal gets placed *outside* the reachable workspace, and a controller simply cannot push the cube to a spot that is off the table. This underscores fundamental design flaws in T2. And T4 regresses slightly, since there the controller already reaches whatever goal it is given, so the loss is entirely in the language and not in the control. So the takeaway is that controller adaptation can cover for a consistent metric bias, but not for an out-of-workspace target or an intent that was never specified enough. This gives us a fairly clean line between what is a grounding problem and what is a control problem.

Our contributions are:

- An **instruction-tier hierarchy** (T0–T4) that works as a reusable lens for evaluating LLM spatial grounding, where we define the difficulty by the kind of reasoning a tier asks for, not by how the sentence is phrased on the surface.
- An **empirical look** at where a frontier LLM’s grounding actually breaks on a continuous-control manipulation task. The accuracy does drop as the abstraction goes up, but our main finding is that this drop is not the interesting part. Each tier breaks in a different way, and the way it breaks is what really matters.
- A **per-tier failure-mode taxonomy** with measured distributions (coordinate drift, region misidentification, out-of-workspace, refusal, hallucination), plus a measurement of how the grounding error spreads into downstream task success under a fixed controller.
- A **controller-adaptation recovery study** showing that downstream adaptation can improve the policy successfully but the scope of this is limited, which lets us draw a line between the grounding problem and the control problem.

We don’t claim the decoupled pipeline idea itself is new [Shah et al., 2023, Ichter et al., 2023, Chevalier-Boisvert et al., 2019]; what we contribute is the structured evaluation and what it ends up showing. Everything here is in simulation, we report uncertainty because the per-tier samples are small ($n = 20$ per tier), and we aren’t dealing with sim-to-real transfer in this work.

2 Related Work

A lot of recent work in language-conditioned robotics uses language models as the front end for robot behavior. Systems like SayCan and LM-Nav show that language models can help connect natural-language commands to lower-level robot skills or navigation goals [Ichter et al., 2023, Shah et al., 2023]. These systems are important for our project because they use the same broad idea of separating language understanding from robot execution. The language model decides what should happen, and a separate controller or skill library handles how to do it. Our work uses that same decoupled structure, but we focus on a smaller failure point. Instead of asking whether the model can choose a whole skill or plan, we ask whether it can turn spatial language into the right continuous target coordinate.

Grounded language learning benchmarks also study how agents follow instructions in controlled environments. BabyAI is one example where agents must connect language to actions in a simulated world [Chevalier-Boisvert et al., 2019]. These kinds of environments are useful because they make language grounding measurable and repeatable. However, many of them are based on discrete states, symbolic objects, or grid-like navigation. Our setting is different because FetchPush-v4 is a continuous manipulation task. The output of the grounder is not just an action label or a grid cell. It is a metric target on the table, so a small spatial error can still matter for the final robot behavior.

Our work is also related to goal-conditioned reinforcement learning. In goal-conditioned control, the policy is trained to reach a provided goal rather than solve only one fixed task. Hindsight Experience Replay is useful in this setting because it lets the policy learn from failed attempts by relabeling what the agent actually achieved as the goal [Andrychowicz et al., 2017]. SAC is also a common choice for continuous-control problems because it learns a stochastic policy while balancing reward and entropy [Haarnoja et al., 2018]. In our pipeline, this matters because the SAC+HER policy can take a coordinate from the language grounder and try to push the block there.

The main difference is that standard goal-conditioned control usually assumes the goal is already correct. In our setting, the goal comes from language, so it can be wrong in different ways. It might be slightly shifted, outside the reachable workspace, or under-specified by the instruction itself. This is why we do not only report one overall success number. We split instructions into tiers from literal coordinates to functional intent and look at what kind of error happens at each tier. This gives us a clearer view of what is actually breaking: sometimes the controller is fine and the language grounding is wrong, while other times the goal itself is too ambiguous to treat as a single coordinate.

3 Background

3.1 Environment (FetchPush-v4)

We run our experiments in FetchPush-v4, a MuJoCo robotic manipulation environment where a Fetch robot arm has to push a block to a target position on a table. The task is useful for our study because it is simple enough to isolate the grounding problem, but still continuous enough that coordinates matter. The agent does not just choose from a small list of discrete actions. It has to move the block in a physical workspace, and the final position of the block is evaluated against a target.

In our setup, the language model outputs a 3D goal coordinate for the block. The target is on the table surface, so the z value is fixed at about 0.42 m. The reachable table region is limited to

$$x \in [1.19, 1.49], \quad y \in [0.60, 0.90].$$

These bounds are important because some language instructions can push the predicted target close to, or slightly outside, the workspace. This happens especially for relative instructions like “move it 5 cm to the right” when the block already starts near an edge. In those cases, the issue is not always that the controller is weak. Sometimes the language-grounded target is simply not a good reachable goal.

FetchPush-v4 also gives us a clean way to separate grounding from control. The grounder decides where the block should go, and the policy decides how to push it there. Because of this separation, we can check whether a failure came from the predicted coordinate or from the physical execution of the push.

3.2 Policy (SAC + HER)

The controller in our system is a goal-conditioned policy trained with SAC and HER. SAC, or Soft Actor-Critic, is a reinforcement learning method designed for continuous-control tasks. It learns a policy that tries to maximize reward while also keeping enough randomness in the actions during training. This makes it a good fit for robotic control problems where the action space is continuous.

HER, or Hindsight Experience Replay, helps the policy learn in sparse-reward goal-reaching tasks. In FetchPush-v4, the robot only gets clear success when the block reaches the target. Many early attempts fail, but HER makes those failures useful by treating the position the block actually reached as if it had been the intended goal. This gives the policy more useful training examples and helps it learn how to reach many different target positions.

For our experiments, the policy takes a target coordinate as input and tries to push the cube to that coordinate. This is important because the same trained policy can be used with many different instructions. The LLM or regex baseline produces the goal, and the SAC+HER policy tries to execute it. We keep this controller fixed in the main evaluation so that changes in end-to-end success mostly reflect the quality of the grounding rather than changes in the robot policy.

This setup also lets us compare two different questions. First, did the language grounder produce the correct coordinate? Second, if it produced a coordinate, could the policy actually reach it? When grounding is correct, the policy usually succeeds. When grounding is wrong, the controller may still reach the predicted goal, but the full task can fail because the predicted goal was not the true intended target.

4 Methods

Our system has two parts. First, a *language grounder* reads a natural-language instruction and outputs a 3D target coordinate on the table. Second, a *goal-conditioned policy* physically pushes the cube to that coordinate. We describe both parts below, along with how we built the instruction set and how we measured success.

4.1 Instruction Library (T0–T4)

We created 100 instructions split evenly across five tiers of 20 each. Each tier tests a different kind of reasoning. All target points sit on the table surface ($z = 0.42$ m) inside the robot’s reachable area ($x \in [1.19, 1.49]$ m, $y \in [0.60, 0.90]$ m).

T0 (Literal Coordinates). The instruction gives the exact coordinates directly (e.g., “push to (1.30, 0.85)”). The grounder just needs to read the numbers. No reasoning required.

T1 (Named Regions). The instruction names a region of the table, like “upper-left corner” or “center.” Each region maps to one of five fixed points (the four corners and the center).

T2 (Relative Offsets). The instruction describes a direction and distance from wherever the cube currently is (e.g., “push it 5 cm to the right”). Before grounding, the harness checks the cube’s actual live position and computes the target from there. This makes T2 the most stochastic tier: since the cube spawns in a different spot each run, the target moves too. If the cube starts near a workspace edge, the offset can push the target out of the area where the policy works well, adding noise that has nothing to do with grounding quality. We flag this issue when discussing T2 results.

T3 (Reference Object Grounding). The instruction refers to an object on the table (e.g., “push it next to the marker”). The grounder is told the positions of five objects (marker, cup, can, pen, cone) and must figure out which one the instruction means. The correct target is the named object’s position.

T4 (Functional Intent). The instruction describes a goal without naming any location (e.g., “get it out of the way,” “put it somewhere safe”). Since there is no single right answer, three team members each picked a reasonable target for every instruction. The official ground truth is the average of those three picks, and the success window is set to the average spread between the annotators plus a 0.03 m buffer. We locked in all T4 ground-truth targets before running any LLM evaluation so the annotations could not be influenced by model outputs.

4.2 LLM Grounder

We use `gemini-3.1-flash-lite` as the grounder, always run at temperature 0 so outputs are deterministic. Every query includes a description of the workspace (axis directions, coordinate limits) plus the instruction and any relevant context. We test three ways of prompting the model:

Zero-shot. The model gets only the workspace description and the instruction. No examples.

Few-shot. Three example instruction–answer pairs are added before the target instruction, one each from T0, T1, and T2. We leave out T3 and T4 examples because their context is too complex to summarize briefly without skewing the output.

Chain-of-thought (CoT). The model is asked to explain its reasoning step by step before giving a coordinate. This costs roughly $10\times$ more tokens than zero-shot.

The model always returns a JSON object. Our parser handles common formatting quirks (code fences, missing z values). If the output cannot be used, we label it with one of four failure modes: *refusal* (the model declined to answer), *parse error* (no valid JSON found), *hallucination* (the coordinate is more than 0.5 m outside the workspace on any axis), or *out-of-workspace* (small drift, within 0.5 m of a boundary).

Regex baseline. We also test a simple rule-based grounder for T0–T2: it pulls coordinate numbers directly from the text (T0), matches region keywords to fixed points (T1), or extracts a distance, unit, and direction and applies the offset (T2). It does not handle T3 or T4. This gives us a baseline to check whether a language model adds any value over pattern matching.

4.3 Evaluation Harness

For each instruction we record three numbers.

Grounder accuracy. Is the predicted coordinate close enough to the true target? We use a 5 cm window for T0–T3 and the annotator-derived window for T4.

Policy success. We pass the predicted coordinate to the SAC+HER policy and run three episodes of up to 50 steps each. An episode counts as a success if the cube reaches that coordinate within FetchPush-v4’s built-in 5 cm threshold at any point during the episode.

End-to-end (E2E) success. Same as policy success, but we check whether the cube reached the *ground-truth* target, not just the predicted one. If the grounder failed to produce a coordinate, all three

Table 1: Baseline per-tier success rates and mean grounder distance. Every cell is $n = 20$ instructions per tier per configuration; policy and E2E rates rest on 60 episodes ($20 \text{ instructions} \times 3 \text{ rollouts}$). The regex baseline returns `unsupported_tier` for T3/T4 by design. Tier tolerances: 5 cm (T0–T2), 8 cm (T3; see text for why), and annotator-derived (T4).

Tier	Grounder	Grounder SR	Policy SR	E2E SR	Mean dist. (m)
T0 – Literal coords	Regex	100.0%	100.0%	98.3%	0.000
	LLM zero-shot	100.0%	100.0%	98.3%	0.000
	LLM few-shot	100.0%	100.0%	98.3%	0.000
	LLM CoT	100.0%	100.0%	98.3%	0.000
T1 – Named regions	Regex	100.0%	96.7%	93.3%	≈ 0
	LLM zero-shot	100.0%	96.7%	93.3%	0.000
	LLM few-shot	100.0%	96.7%	93.3%	0.000
	LLM CoT	100.0%	96.7%	93.3%	0.000
T2 – Relative offsets	Regex	100.0%	95.0%	85.0%	≈ 0
	LLM zero-shot	40.0%	95.0%	68.3%	0.049
	LLM few-shot	45.0%	95.0%	76.7%	0.034
	LLM CoT	65.0%	96.7%	60.0%	0.032
T3 – Reference objects	Regex	0.0%	—	0.0%	—
	LLM zero-shot	100.0%	96.7%	71.7%	0.060
	LLM few-shot	100.0%	96.7%	73.3%	0.060
	LLM CoT	100.0%	96.7%	78.3%	0.059
T4 – Functional intent	Regex	—	—	—	—
	LLM zero-shot	55.0%	96.7%	55.0%	0.186
	LLM few-shot	50.0%	93.3%	50.0%	0.193
	LLM CoT	45.0%	96.7%	45.0%	0.195

episodes are automatically counted as failures. This metric captures the full pipeline: a grounder error cannot be rescued by a good controller.

We report all three metrics broken down by tier and prompt variant.

5 Experiment & Results

The whole point of this first experiment is to measure the grounding on its own. We train the SAC+HER policy once on FetchPush-v4 and then freeze it, and every grounder configuration feeds goals into that same policy, so any difference in end-to-end success comes from the grounding and not the control. We run all 100 instructions (20 per tier, T0–T4; Section 4.1) through five configurations: the regex baseline (T0–T2 only) and the `gemini-3.1-flash-lite` grounder under zero-shot, few-shot, and chain-of-thought (CoT). Each instruction gets three episodes, so every per-tier policy and end-to-end number is over 60 episodes.

We report three numbers for each configuration, all defined in Section 4.3. Grounder SR is the fraction of instructions where the predicted goal lands within the tier’s tolerance of the ground-truth goal. Policy SR is the fraction of rollouts where the cube reaches the predicted goal within the environment’s native 5 cm tolerance, so it tells us how good the control is given whatever goal it was handed. E2E SR is the fraction of rollouts where the cube reaches the ground-truth goal within the tier tolerance, so it covers the whole pipeline. The tolerances are 5 cm for T0–T3 and a per-instruction annotator-derived value for T4 (0.064–0.078 m).

5.1 Baseline Evaluation

At T0 and T1 the LLM and the regex baseline are basically the same: both hit 100% grounder SR with the same policy and E2E rates (98.3% at T0, 93.3% at T1). That’s what we expected, since T0 is just parsing coordinates and T1 is a lookup against the shared region table, but it’s a useful baseline to have. The LLM adds nothing where a simple rule already works, so any gap between the LLM and

regex at T2 and up is real and not just noise. Grounding is basically solved at both tiers, and the little bit of E2E that’s missing is on the policy side, not the grounder.

T2 is where grounding first breaks, and it breaks in a way we didn’t expect. The regex baseline parses the offsets perfectly (100% grounder SR), but the LLM only gets 40–65% of T2 goals within tolerance, so the language model is actually worse than a plain rule at applying a fixed offset. This is the T2 cliff. It isn’t a formatting problem, since Section 5.2 shows no malformed or refused outputs at T2, it’s a placement problem: the mean grounder distances (0.032–0.049 m) are small, but they’re still enough to push past the 5 cm tolerance.

The interesting part is what happens when you look at grounder SR and E2E SR together, because the three variants don’t line up the way you’d think. CoT has the best T2 grounder SR (65% vs 40–45%) but the worst T2 E2E (60.0% vs 68.3% and 76.7%), and few-shot has the best T2 E2E (76.7%) even though its grounder SR is lower than CoT’s and its policy SR is a bit lower too (95.0% vs 96.7%). The reason, which we get into in Section 5.2, is the kind of error each variant makes. Zero-shot and few-shot mostly fail by putting a correctly-aimed goal just past the edge of the workspace, but since the policy then pushes the cube up to that edge, and the edge is within the E2E tolerance of the off-table goal, a lot of these still count as end-to-end successes. That’s why T2 E2E actually comes out higher than grounder SR for both of them. CoT does the opposite: it keeps the goal inside the workspace but flips the sign of the offset, so the cube ends up on the wrong side of where it started, and there’s no way for the policy to recover from that. So CoT turns a higher grounder SR into a lower E2E SR. This is the first clear case of what the whole paper is about: it’s the kind of grounding error that decides end-to-end success, not how often the grounder is off.

T3 is scored at an 8 cm tolerance rather than the 5 cm we use for T0–T2, and the reason matters. As we show in Section 5.2, the grounder puts the goal almost exactly 6 cm from the named object every single time (mean distance 0.059–0.060 m). Phrases like “next to” and “near” do not pin down a distance, so at a 5 cm window every T3 case fails on that fixed offset alone and we would be measuring the particular ground truth we happened to pick rather than the grounder’s ability. At 8 cm the grounder reaches 100% across the board and T3 E2E lands at 71.7–78.3%, best with CoT. T4 is the floor: grounder and E2E SR are both stuck around 45–55% and move together, because the policy reaches just about any goal you give it, and no prompting variant gets out of that range. The differences between variants at T4 (≤ 5 pp on grounder SR, ≤ 10 pp on E2E) are smaller than how much the human annotators disagreed with each other on the same instructions, so we treat T4 as basically a coin flip no matter how you prompt.

One more check: when the grounding is right, the policy almost always reaches the goal. Conditioning on a correct grounding, mean policy SR is 1.000 (T0), 0.967 (T1), and 0.949–1.000 (T2, depending on variant). So control isn’t the bottleneck anywhere the grounding works, which is the whole reason we can read E2E SR as a measure of grounding quality. And with only 20 instructions per tier, we treat any gap smaller than about 10 pp as suggestive, not solid.

5.2 Failure Analysis

The classifier from Section 4.2 tags each output as *refusal* (the model declines or hedges), *parse error* (no usable JSON), *hallucination* (goal more than 0.5 m outside the workspace), or *out-of-workspace* (OOW; goal just outside a bound, within 0.5 m of it), and the regex baseline also returns *unsupported_tier* for T3 and T4. A grounding can also miss without getting any of these tags: the goal parses fine and sits inside the workspace, but it’s still too far from the ground truth, which we call an *in-bounds error*. T0 and T1 don’t fail at all under any variant, so we only look at T2–T4 here. And across every tier and variant, refusals, parse errors, and hallucinations are all zero (Table 2), so every failure is either OOW or in-bounds.

T2: out of bounds vs. flipped signs. Two things are going wrong at T2 at the same time. The first is boundary clipping (these get tagged OOW): the model picks the right axis and direction, but the cube’s current position plus the offset ends up past the edge of the workspace. For T2_001, “push it 5 cm to the right” with the cube at $y \approx 0.616$, the grounder returns [1.469, 0.566, 0.425], and that y of 0.566 is below $y_{\min} = 0.60$. The grounding is right, the workspace is just too tight to fit the offset. This is 8 of the failures for both zero-shot and few-shot, and it never happens under CoT. The second is a flipped sign: the model uses the right axis but the wrong direction. For “move the block 7 cm to the left” it returns $y = 0.825$ when the ground truth is $y = 0.965$, which is 0.140 m off, exactly twice

Table 2: Per-tier failure-mode counts for the LLM grounder (out of 20 instructions). The *Other* column aggregates hallucination, refusal, and parse error, which are zero throughout. T0, T1, and T3 carry no failure tags and are omitted (T3 reaches 100% grounder SR under an 8 cm tolerance; see text).

Tier	Variant	OOW	In-bounds error	Other	Total fail
T2 – Relative offsets	Zero-shot	8	4	0	12
	Few-shot	8	3	0	11
	CoT	0	7	0	7
T4 – Functional intent	Zero-shot	0	9	0	9
	Few-shot	0	10	0	10
	CoT	0	11	0	11

the 7 cm offset, so it went $-y$ (right) instead of $+y$ (left) even though the prompt says left is $+y$. This one barely shows up in zero-shot and few-shot (4 and 3 cases) but it’s all seven of the CoT failures. So CoT basically swaps one problem for the other: its reasoning step does catch the boundary cases (no OOW), but the in-bounds goals it puts down instead land the cube on the wrong side of where it started, and the policy can’t fix that. That’s why CoT’s better grounder SR turns into a worse E2E SR.

T3: the same 6 cm every time. T3 has a 100% grounder SR, but if you look at what it’s doing it’s almost mechanical. On all 60 T3 cases the goal sits almost exactly 6 cm from the named object: the min, max, and mean offset are all 0.060 m. It’s almost always along y (19 of 20) and only once along x (T3_016, “place the box near the pen,” shifts $+0.06$ in x), and the sign just follows whichever object is named, $+y$ for 11 of them and $-y$ for the other 9. So the grounder gets the right object and a reasonable direction, but then it applies one fixed offset that it pulled from the workspace description, not from the instruction. Under the 5 cm tolerance used for T0–T2 every T3 case would fail on that offset alone, which is why the tier is scored at 8 cm. And that consistency is the point: it’s a systematic bias, not random scatter, which is exactly the thing Section 6 takes advantage of.

T4: disagreement with the annotators. At T4 every failure is in-bounds. The model doesn’t refuse, doesn’t break the JSON, and doesn’t go outside the workspace, it just doesn’t agree with where the annotators put the goal. And the gaps are big. From zero-shot: “get it out of the way” gives $[1.19, 0.60, 0.42]$, the lower-right corner, while the annotators settled on $[1.43, 0.73, 0.42]$ in the mid-right (0.27 m apart); “move it as far back as possible” gives $[1.19, 0.75, 0.42]$ at the $x_{\min}$ edge versus $[1.44, 0.80, 0.42]$ (0.26 m), with the model and the annotators reading “back” as opposite directions; “move it somewhere it won’t get knocked over” gives $[1.49, 0.90, 0.42]$, the upper-left corner, versus $[1.46, 0.63, 0.42]$ in the lower-right (0.27 m). The pattern is pretty clear: when the instruction is vague the model just dumps the cube in a corner, and the annotators don’t. Mean grounder distance barely moves across variants (0.186–0.195 m) and it’s about the same as how far apart the annotators are from each other (around 0.20 m), so this is a problem with the instruction itself, not with the prompt, and you’re not going to fix it by prompt engineering.

Putting it together, the four tiers that actually fail each fail in their own way. T0 and T1 don’t fail. T2 is a mix of boundary clips (zero-shot and few-shot) and sign flips (CoT), and CoT just trades one for the other without making the total any better. T3 fails due to the ambiguity of “near” style commands, underneath that it’s putting the same 6 cm offset on every instruction, which is a systematic bias and not random noise. T4 fails entirely in-bounds, with the model parking the cube in corners and the annotators not, and the prompt barely matters. So we end up with three different kinds of error: a consistent offset at T3 that a controller could in principle absorb, an off-table or flipped goal at T2 that it can’t, and an underspecified target at T4 that isn’t really a control problem at all. That’s the question Section 6 picks up, which of these the policy can actually be retrained around.

6 Retraining

The goal of retraining was to explore whether we could push the threshold of where language-grounded instructions break for the policy without changes to the grounding. Failures in T2 and T3 were largely identified as grounder errors, but there is room for improvement in the policy itself in

T2, T3, and T4. We set out to identify a retraining plan to increase the goal robustness of the policy in those tiers specifically. We examined the failure points of each tier to build a more adversarial distribution of goals for retraining. Additionally, we opted to use a shaped reward over fetch-push’s binary reward to address proximity issues in T3. This resulted in a 3-phase retraining plan outlined below.

6.1 Goal Distribution Augmentation

The majority of failures identified in T2 instructions are the result of out-of-bounds or near-edge-of-workspace/boundary issues. We observed that for T2 instructions, CoT prompting performs the best with a grounder success rate of 65%, but the entire pipeline struggles with this process, which is not surprising and discussed previously in section 5.2. To address this, we propose to retrain the policy with a biased sampling of 30% towards near-edge goals. Concretely, this will look like biasing 30% of the HER goal sampling towards goals within 5 cm of the edge. The hypothesis is that the policy will become more robust/confident with pushing the cube to targets at the edge of the workspace.

Additionally, this same principle can be used to improve T4; we observed that the grounder handled T4 instructions by defining “corners” of the workspace to handle dealing with obscure instructions like “out of the way.” The LLM picks whichever corner it thinks works best and sends the cube there, so success is a coin flip. In an attempt to work with the LLM rather than against it, we will retrain with an additional bias towards the corners of the workspace as well, so that the policy also becomes more confident in dealing with workspace corners. Concretely both of these will look like a 20/10 bias split; 20% of HER goal sampling towards goals within 5 cm of the edge, and 10% toward workspace corners.

6.2 Shaped reward for T3 offset correction

The 5 cm threshold causes nearly all T3 instructions to fail due to the LLMs caution in its placement of the cube. The grounder is consistently predicting a goal 6 cm off on the Y-axis based on our results. FetchPush-v4 by default utilizes a sparse binary reward function, so we will augment this function with distance-proportionality shaping. This will penalize runs that aren’t as close to the target and encourage getting closer to the target. This is to push the policy to find the right region, even if the goal is slightly off. We keep α small, 0.1, so the policy learns how close it got, ideally. Concretely, for episodes where the cube is more than 4 cm from the goal during retraining, add a distance-proportionality penalty to the reward:

$$r_{\text{shaped}} = r_{\text{sparse}} + \alpha * (1 - d_{\text{block_to_true_goal}} / d_{\text{threshold}})$$

6.3 Targeted Replay on failure cases

As a “catch-all” strategy, we take all failure cases from T2/T3 runs and inject them as seeds for the policy to retrain on at a higher frequency. This should create an oversampling of “harder” cases for the policy. We inject these “harder” cases in training early so that these seeds are seen by the policy in the early stages of training, leading to better adaptation. The original experiment recorded: predicted goal, ground truth goal, and policy episodes, all of which will be used for seeding.

7 Post-Retraining Results

The retrained policy was substituted for the original policy in the evaluation harness, and the same 100 instructions were evaluated by the harness again. The grounder SR didn’t change after retraining as expected since only the policy was changed. The policy SR improved to 100% across the board, and the E2E SR showed improvement predominantly across T0/T1/T2. For T3 in Table 3, E2E is reported under the 5 cm threshold, so it appears near zero even though the policy reaches the LLM-predicted goal.

7.1 Retraining Failure Analysis

Regression only occurs in two places; T2 CoT E2E SR dropped from 60.0% to 53.3%, and T4 Zero-shot/Few-shot prompting’s E2E SR dropped by 3.3%. As previously discussed in section 5.2, T2’s E2E SR is inherently noisy because ground truth is recomputed from a stochastic cube spawn

Table 3: Post-retraining evaluation results across all tiers and prompt variants. Δ E2E denotes change from baseline.

Tier	Variant	Grounder SR	Policy SR	E2E SR	Δ E2E
T0 – Literal Coordinates	Zero-shot	100.0%	100.0%	100.0%	+1.7%
	Few-shot	100.0%	100.0%	100.0%	+1.7%
	CoT	100.0%	100.0%	100.0%	+1.7%
T1 – Named Regions	Zero-shot	100.0%	100.0%	100.0%	+6.7%
	Few-shot	100.0%	100.0%	100.0%	+6.7%
	CoT	100.0%	100.0%	100.0%	+6.7%
T2 – Relative Offsets	Zero-shot	40.0%	100.0%	75.0%	+6.7%
	Few-shot	45.0%	100.0%	80.0%	+3.3%
	CoT	65.0%	100.0%	53.3%	-6.7%
T3 – Reference Object Grounding	Zero-shot	100.0%	100.0%	0.0%	+0.0%
	Few-shot	100.0%	100.0%	0.0%	+0.0%
	CoT	100.0%	100.0%	5.0%	+0.0%
T4 – Functional Intent	Zero-shot	55.0%	100.0%	51.7%	-3.3%
	Few-shot	50.0%	100.0%	46.7%	-3.3%
	CoT	45.0%	100.0%	45.0%	+0.0%

position each time. This was an overlooked limitation in the design of this project and T2’s evaluation specifically. However, this means this is not in fact regression, but rather expected variance in T2 E2E SR evaluation.

For the apparent T4 regression, the same instruction in both zero-shot and few-shot variants was the culprit, instruction T4_017. Upon further examination this issue is attributed to design flaws in the T4 evaluation. T4 evaluation is inherently limited by the chosen method of evaluating against annotator consensus, the policy and grounder did everything right here, pushing the cube to a corner. The corner, however, was not one agreed on by the annotators, causing this to appear as a fail when in fact the policy behaved correctly.

7.2 Evaluation of Improvements

The initial T0 E2E SR of 98.3% and T1 E2E SR of 93.3% were both “cleaned up” by the retrained policy, closing both of those gaps and reaching a 100.0% SR for both across all prompt variants. Looking more closely, T0 had failed E2E on one instruction, T0_019, which prompts to move the cube to the center of the workspace, requiring precise movement of the policy. The retrained policy was able to handle this case. The more notable result lies in T1, the initial policy failed E2E on: T1_003, T1_004, T1_006, and T1_016; all of which involve corners (e.g. T1_003 = "move the cube to the upper-left corner"). The retrained policy had 100.0% SR in these 4 instructions and this can be attributed to the goal distribution augmentation, specifically the corner biasing which clearly made the policy more robust in its ability to handle corners of the workspace.

While there are observed increases in the E2E SR of T2 zero-shot and few-shot variants, the aforementioned problems with T2 evaluation can be attributed to the increased SR. Once again this is expected variance due to design limitations and cannot be considered an improvement.

Reference Object Grounding, T3, once again is misleading in its results. The grounder and policy SR are both 100.0% yet the E2E SR is near zero. As discussed in section 5.1 the policy and grounder are actually succeeding, however, the grounder just always places the cube with a systematic ~ 6 cm bias in the Y direction. The policy and grounder are working just not within the 5 cm threshold which was the motivation for adding shaped reward. T3’s E2E SR remained steady in the retrained evaluations however, upon further examination there are active improvements under the hood. Comparing the mean policy distance of the original policy to the retrained policy reveals that the shaped reward is working. Table 4 showcases that the desired behavior of adding shaped reward is working across all variants. The policy is getting the cube closer to the grounder’s predicted goal more precisely in the retrained policy. However, the +6 cm Y-direction bias persists in the grounder still. Across all

Table 4: Mean policy distance measures how close the cube gets to the grounder’s predicted goal (lower is better).

Variant	Original	Retrained	Improvement
Zero-shot	30.6mm	25.3mm	−5.3mm
Few-shot	30.1mm	25.3mm	−4.8mm
CoT	38.7mm	23.4mm	−15.3mm

instructions the policy is getting the cube closer to the goal, just not close enough to meet the 5 cm threshold that was set. So despite the apparent non-improvement, the policy is actually improving as expected due to the shaped reward change.

The final and arguably most substantial improvement is the fact that across all tiers and variants, the policy SR has reached 100.0%. Prior to retraining the policy on its own was failing even when given valid goals by the grounder. Retraining successfully eliminated all policy-side errors from the pipeline. This effectively isolates and highlights the remaining failure cases to two issues: the grounder and our evaluation metrics.

8 Conclusion

We set out to find where an LLM’s spatial grounding breaks once the language grounded instructions leave literal coordinates. We measured it against a fixed controller so the grounding could be read on its own. As hypothesized, accuracy does decline with abstraction: T0 and T1 are near-perfect, T2 at 40.0–65.0%, T3 near zero under a 5 cm tolerance, and T4 hovering at the 45.0-55.0% range. But the decline is the least interesting thing we found. A single accuracy measurement says how often a tier is wrong; it says nothing about whether that can be recovered downstream, and recoverability is what really decides whether an instruction style is safe to put in front of a user.

Through the lens of recoverability, we can look at each tier beyond just how they score in our metrics. T0 and T1 are solved and need no language model at all; a regex matches the LLM exactly, so the LLM serves no purpose. In T2 the LLM is actively worse than the regex baseline at applying a fixed offset (40–65.0% vs 100.0%), and its errors are unrecoverable: it either places the goal off the table or flips the sign of the offset. No controller can recover from this no matter how perfect the policy is. T3 is encouraging, the grounder resolves the right reference object and the right direction every time. Its only error is the near-constant 6 cm offset in Y, a problem that is recoverable down the line. T4 is not a grounding-accuracy problem at all: the instruction never pins down one target. The "accuracy" ends up measuring our annotations as much as the model, pointing out fundamental flaws in SR metrics in T4.

The second experiment, adapting the controller by retraining the policy, is what lets us truly determine what is/is not recoverable. After retraining, policy SR reached 100.0% across every tier, isolating the remaining failures to the grounder and our metrics. Where the error was a consistent bias (T3), the shaped reward measurably pulled the cube closer to the predicted goal. This is most evident under CoT (38.7 mm → 23.4 mm) confirming that the offset is the kind of thing a controller can be trained toward, even though the 6 cm grounder bias keeps it just outside the 5 cm window. Where the error was structural (T2 off-table goals, T4 underspecification), adaptation did nothing real: the apparent T2 changes are noise from the stochastic cube spawn, and the apparent T4 regression traces to a single instruction the annotators happened to disagree on. Adaptation allowed us to draw a line between a grounding problem and a control problem.

We once again assert that the most important takeaways from these experiments are not necessarily the numerical results but the categorization of the results. A 6 cm offset on 60 out of 60 T3 cases, or a regex beating the LLM on every T2 instance are robust failure cases. The E2E changes from retraining are not robust and with 20 instructions per tier they sit inside the noise (T2/T4). T3 remains partly a definitional problem, since "next to the marker" has no single correct distance, and T4 needs a reworking before its numbers carry weight. Everything here is in simulation, so we make no sim-to-real claim.

The broader lesson for language-driven robotics is that "how accurate is the grounding" is the wrong question to lead with. What matters more is what kind of error a given instruction style produces

and whether that error can be recovered downstream. Concretely: use a rule rather than an LLM for literal and region instructions (T0/T1); do not trust an LLM for relative offsets without clamping or rejecting out-of-workspace goals at the grounding stage (T2); treat reference object grounding as a recalibration problem rather than a failure (T3); and do not score functional intent instructions as if they had one coordinate answer (T4).

References

- Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hind-sight Experience Replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/453fadbd8a1a3af50a9df4df899537b5-Abstract.html.
- Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. BabyAI: A platform to study the sample efficiency of grounded language learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=rJeXCo0cYX>.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 2018. URL <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- Brian Ichter et al. Do As I Can, Not As I Say: Grounding language in robotic affordances. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2023. URL <https://proceedings.mlr.press/v205/ichter23a.html>.
- Dhruv Shah, Błażej Osioński, Brian Ichter, and Sergey Levine. LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 492–504. PMLR, 2023. URL <https://proceedings.mlr.press/v205/shah23b.html>.